


UNIVERSITÉ PARIS-SACLAY

DÉPARTEMENT D'INFORMATIQUE

*M1 CNS – Parcours Systèmes Autonomiques*

*Travail d'Étude et de Recherche*

---

# ALIGNEMENT DES LLMs AVEC LES GRAPHES DE CONNAISSANCES POUR LA GÉNÉRATION D'EXPLICATIONS ADAPTÉES

---

## Présenté par :

MOKHTARI Dhia Elhak
*dhiaelhakmokhtari@gmail.com*

LAKHMI Billal Hichem
*lakhmibilel12@gmail.com*

OUHENIA Anis
*anisouhenia.pro@gmail.com*

**Encadrant :**

HAMIDI Massinissa

Année universitaire 2024–2025

# Table des matières

**1 Introduction** **4**
1.1 Objectif du Travail d'étude et de recherche . . . . . 4

**2 État de l'art** **4**
2.1 Les grands modèles de langage (LLMs) dans l'éducation . . . . . 4
2.2 Représentations mentales en sciences cognitives et éducatives . . . . . 5
2.3 Graphes de connaissances . . . . . 5
2.4 Techniques d'alignement des LLMs . . . . . 6

**3 Méthodologie** **6**
3.1 Construction du graphe de connaissances . . . . . 6
3.2 Modélisation des connaissances de l'apprenant . . . . . 7
3.3 Alignement du LLM pour la génération d'explications adaptées . . . . . 8

**4 Implémentation** **8**
4.1 Technologies et outils . . . . . 8
4.2 Architecture du système . . . . . 9
4.3 Fonctionnalités implémentées . . . . . 10

**5 Expérimentations et résultats** **10**
5.1 Protocole expérimental simulé . . . . . 11
5.2 Résultats observés et comparatifs . . . . . 13
5.3 Analyse qualitative . . . . . 14

**6 Discussion et limitations** **14**
6.1 Points forts . . . . . 14
6.2 Limitations identifiées . . . . . 15

**7 Perspectives** **15**
7.1 Synthèse des contributions . . . . . 15
7.2 Perspectives futures . . . . . 16

**8 Conclusion** **16**

**Références** **17**

**Glossaire** **19**

**Annexes** **20**

**Abstract**

Les grands modèles de langage (LLMs) transforment radicalement les méthodologies pédagogiques grâce à leur capacité à générer des contenus éducatifs dynamiques. Toutefois, leur utilisation optimale en éducation se heurte à une limitation fondamentale : leur incapacité à s'adapter aux représentations mentales individuelles des apprenants et à intégrer leurs prérequis spécifiques. Ce travail propose une approche innovante combinant un graphe de connaissances pondéré et un LLM pour pallier ces lacunes.

Notre système articule deux composantes synergiques :

— Un graphe de connaissances modélisant les relations hiérarchiques entre concepts pédagogiques, enrichi de poids reflétant les dépendances de prérequis.

— Un mécanisme d'orientation du LLM exploitant cette structure pour générer des explications et des quiz adaptés au profil cognitif de l'étudiant.

L'originalité de l'approche réside dans l'alignement dynamique entre la génération de contenu et l'état des connaissances de l'apprenant. Le système identifie les concepts maîtrisés, détecte les lacunes prérequises, et guide le LLM pour produire des réponses pédagogiquement cohérentes et progressives.

Les résultats préliminaires suggèrent une amélioration significative de la pertinence contextuelle des sorties du LLM, notamment dans la création d'explications ciblées et d'exercices d'auto-évaluation adaptatifs. Cette fusion entre raisonnement symbolique (graphe) et apprentissage statistique (LLM) ouvre des perspectives prometteuses pour l'éducation personnalisée à grande échelle.

# 1. Introduction

Les grands modèles de langage (LLMs) révolutionnent actuellement de nombreux domaines, dont celui de l'éducation. Leur capacité à générer du contenu cohérent et pertinent ouvre de nouvelles perspectives pour la création de ressources pédagogiques personnalisées. Cependant, malgré leurs performances impressionnantes, ces modèles présentent encore des lacunes significatives dans leur capacité à s'adapter aux différences individuelles des apprenants.

L'un des défis majeurs dans l'utilisation des LLMs pour l'éducation réside dans leur incapacité native à prendre en compte la représentation mentale unique de chaque apprenant. Cette représentation, forgée par les expériences personnelles et les connaissances préalablement acquises, conditionne fortement la façon dont un nouvel apprentissage sera assimilé. De plus, les LLMs ne sont pas intrinsèquement conçus pour identifier et s'appuyer sur les prérequis spécifiques à chaque apprenant, ce qui limite considérablement la pertinence des contenus générés.

## 1.1. Objectif du Travail d'étude et de recherche

Ce travail vise à concevoir un assistant pédagogique intelligent capable de générer automatiquement des explications et des quiz adaptés au niveau réel de chaque étudiant. Pour cela, nous combinons deux composantes principales :

— Un graphe de connaissances pondéré représentant les relations de prérequis entre concepts.

— Un grand modèle de langage (LLM) capable de produire du contenu éducatif personnalisé.

L'objectif est d'exploiter la structure du graphe pour guider le LLM dans la génération d'explications ciblées, en tenant compte de ce que l'étudiant connaît déjà (`KNOWN`) et des prérequis manquants. Ce système cherche à améliorer l'apprentissage en rendant les réponses du LLM plus cohérentes, progressives et pédagogiquement pertinentes.

# 2. État de l'art

## 2.1. Les grands modèles de langage (LLMs) dans l'éducation

Les LLMs comme GPT-4, Claude ou Mistral ont démontré des capacités impressionnantes dans la génération de texte cohérent et contextuel. Dans le domaine éducatif, ces modèles sont déjà utilisés pour diverses applications :

— Génération automatisée de questions et quiz.

— Création de contenus pédagogiques adaptés à différents niveaux.

— Production d'explications et d'exemples pour illustrer des concepts complexes.

— Assistance à la rédaction et correction de travaux d'étudiants.

Cependant, comme le soulignent [2] et [4], ces applications présentent encore des limitations importantes quant à la personnalisation des contenus. Les LLMs génèrent généralement des réponses basées sur des patterns statistiques sans véritable compréhension du niveau de connaissance spécifique de l'apprenant.




FIGURE 1 – Liste des LLMs les plus utilisés en 2024 (Source : Tech Radar)

## 2.2. Représentations mentales en sciences cognitives et éducatives

Les sciences cognitives ont largement étudié la façon dont les connaissances sont organisées dans l'esprit humain. Plusieurs théories et modèles sont particulièrement pertinents pour notre étude :

— **Les cartes conceptuelles** : Développées par Novak et Gowin [5], elles représentent graphiquement les relations entre les concepts et aident à visualiser la structure de connaissance d'un individu.

— **La théorie des schémas** : Proposée initialement par Bartlett [6] puis développée par d'autres chercheurs, elle suggère que les connaissances sont organisées en structures cognitives appelées schémas, qui facilitent la compréhension et l'assimilation de nouvelles informations.

— **Les modèles mentaux** : Johnson-Laird [7] décrit comment les individus construisent des représentations internes des phénomènes qu'ils observent, influençant leur raisonnement et leur apprentissage.

Ces théories mettent en évidence l'importance d'identifier et d'exploiter les structures de connaissances préexistantes chez l'apprenant pour faciliter l'acquisition de nouvelles connaissances.

## 2.3. Graphes de connaissances

Les graphes de connaissances constituent une approche prometteuse pour modéliser formellement les structures de connaissances. Ils permettent de représenter les concepts et leurs relations sous forme de graphes orientés [8]. Dans le contexte éducatif, plusieurs travaux ont exploré l'utilisation des graphes de connaissances :

— **Knowledge Forest** [9] : Utilise des graphes pour représenter les structures de connaissances et guider la génération de contenu pédagogique.

— **Intelligent Tutoring Systems** : Des systèmes comme AutoTutor [10] intègrent des représentations de connaissances pour adapter leurs interactions aux besoins spécifiques des apprenants.

— **Curriculum Learning** : L'organisation de l'apprentissage selon une progression logique basée sur les prérequis conceptuels [11].

Ces approches démontrent le potentiel des graphes de connaissances pour structurer et personnaliser l'apprentissage, mais leur intégration avec les capacités des LLMs reste un domaine relativement inexploré.

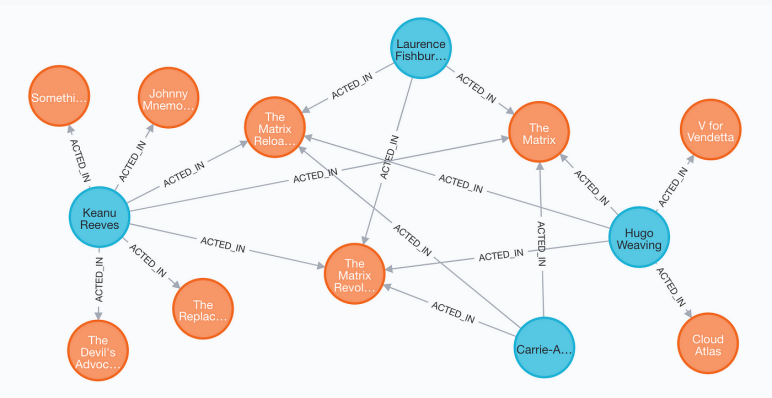



FIGURE 2 – Exemple d'un graphe de connaissances

### 2.4. Techniques d'alignement des LLMs

L'alignement des LLMs vise à faire correspondre leur comportement avec des objectifs ou contraintes spécifiques. Plusieurs techniques ont été développées récemment :

— **Prompt Engineering** : Manipulation des instructions fournies au modèle pour orienter sa génération [12].

— **Fine-tuning** : Ajustement des paramètres du modèle sur des données spécifiques au domaine ou à la tâche [13].

— **RLHF** (Reinforcement Learning from Human Feedback) : Utilisation de feedback humain pour affiner le comportement du modèle [14].

— **Constitutional AI** : Approche visant à aligner les modèles sur des principes ou contraintes spécifiques [15].

Dans le contexte éducatif, des travaux comme ceux de [16] ont commencé à explorer comment aligner les LLMs avec des objectifs pédagogiques, mais l'intégration avec des représentations individuelles des connaissances reste un défi majeur.

## 3. Méthodologie

Notre approche méthodologique s'articule autour de trois axes principaux : la construction d'un graphe de connaissances représentant les concepts du domaine et leurs relations, l'évaluation des connaissances de l'apprenant, et l'alignement du LLM pour générer des explications adaptées.

### 3.1. Construction du graphe de connaissances

Le graphe de connaissances constitue la structure fondamentale représentant le domaine d'apprentissage et les relations entre les concepts. Sa construction repose sur une approche semi-automatique combinant extraction automatique et validation humaine.

Notre méthodologie s'appuie sur l'algorithme ACE (AI-Assisted Construction of Educational Knowledge Graphs with Prerequisite Relations) [3], qui permet de générer des graphes orientés, pondérés et acycliques à partir de contenus pédagogiques textuels. Les étapes suivies sont les suivantes :

— **Extraction des concepts et des paires candidates** : Identification des concepts fondamentaux par analyse de curricula, manuels et ressources pédagogiques, suivie de la génération automatique de paires de concepts

via ACE.

— **Évaluation sémantique des relations** : Chaque paire de concepts est évaluée selon deux scores :
  — **CSR** (Concept Semantic Relatedness) : Mesure la proximité sémantique des concepts dans le texte.
  — **CER** (Concept Embedding Relatedness) : Évalue leur proximité vectorielle dans un espace d'embedding.
— **Filtrage et pondération des relations** : Les paires obtenant les scores les plus élevés sont sélectionnées comme relations de type *PREREQUISITE*, avec un poids compris entre 0 et 1 reflétant l'intensité du lien.
— **Enrichissement par LLM** : Utilisation d'un LLM pour suggérer des concepts ou relations supplémentaires, validés manuellement.
— **Validation et raffinement** : Vérification de la cohérence globale du graphe et de la pertinence des relations par des experts du domaine.

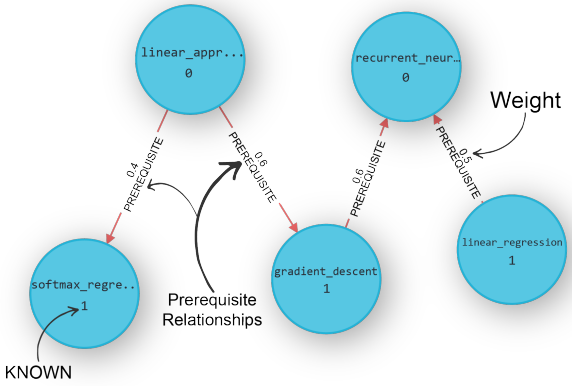



FIGURE 3 – Architecture du graphe de connaissances

## 3.2. Modélisation des connaissances de l'apprenant

Pour adapter les explications au niveau de chaque apprenant, nous devons évaluer ses connaissances actuelles, en tenant compte de la force des relations entre les concepts dans le graphe de connaissances ainsi que de l'état de maîtrise de chaque concept. Chaque concept dans le graphe est marqué avec un flag `KNOWN`, qui peut avoir les valeurs suivantes :

— **0** : Le concept est inconnu.
— **1** : Le concept est maîtrisé.

En parallèle, les relations de prérequis entre les concepts sont associées à des poids compris entre 0 et 1, représentant la force de la dépendance entre les concepts. Ces poids permettent au système de prioriser les prérequis les plus importants dans le processus d'explication.

Notre méthodologie comprend les étapes suivantes :

— **Évaluation diagnostique initiale** : Le système interroge le graphe de connaissances pour obtenir la liste des concepts inconnus (`KNOWN = 0`). Chaque concept est présenté à l'apprenant pour évaluer sa maîtrise. Si un concept est jugé maîtrisé (`KNOWN = 1`), il est mis à jour dans le graphe.
— **Annotation du graphe** : Le flag `KNOWN` est utilisé pour marquer le niveau de maîtrise de chaque concept

(soit 0 pour inconnu, soit 1 pour maîtrisé). Lorsqu'un concept est maîtrisé, les concepts liés par des prérequis avec des poids élevés sont considérés comme plus urgents à aborder dans l'explication, tandis que ceux avec un poids faible peuvent être traités plus tard ou de manière optionnelle.

— **Mise à jour dynamique** : L'état de connaissance de l'apprenant est mis à jour à chaque interaction. Par exemple, un concept avec un flag `KNOWN` égal à 0 sera lié à des concepts ayant des prérequis avec des poids différents. Les prérequis avec un poids plus élevé seront prioritaires dans les explications, et le flag `KNOWN` des concepts sera ajusté en fonction des performances de l'apprenant.

— **Inférence de connaissances** : Le système utilise des règles d'inférence pour estimer la maîtrise des concepts non directement évalués. L'inférence prend en compte la valeur `KNOWN` des concepts liés et le poids des relations pour estimer si un concept non évalué est effectivement connu. Un concept avec un poids de relation élevé pourra être traité plus tôt, même s'il n'a pas encore été directement évalué.

### 3.3. Alignement du LLM pour la génération d'explications adaptées

L'objectif central est d'adapter les explications générées par le LLM au niveau de connaissance spécifique de l'apprenant. Notre approche d'alignement comprend :

— **Construction de prompts contextualisés** : Intégration des informations du graphe de connaissances et du profil de l'apprenant dans les instructions fournies au LLM.

— **Génération multi-étapes** : Utilisation d'une approche séquentielle où le LLM planifie d'abord l'explication en fonction des prérequis identifiés, puis génère le contenu correspondant.

— **Post-traitement basé sur les règles** : Application de règles de vérification pour s'assurer que l'explication n'utilise pas de concepts inconnus sans les expliquer.

— **Fine-tuning ciblé** : Adaptation du modèle sur un corpus d'explications annotées selon différents niveaux de connaissances préalables.

## 4. Implémentation

Notre implémentation repose sur une architecture modulaire intégrant plusieurs composants techniques. Cette section détaille les technologies utilisées et les choix d'implémentation.

### 4.1. Technologies et outils

Pour la réalisation de notre système, nous avons sélectionné les technologies suivantes :

— **Base de données de graphe Neo4j** : Pour le stockage et la manipulation efficace du graphe de connaissances, permettant des requêtes complexes sur les relations entre concepts.

— **LLM local (Mistral)** : Déployé via Ollama pour permettre une génération de contenu flexible et contrôlable, tout en limitant les coûts et les problèmes de confidentialité.

— **Framework Python** : Utilisation de bibliothèques comme LangChain pour faciliter l'interaction avec le LLM et la construction de chaînes de traitement.

— **Interface utilisateur Chainlit** : Création d'une interface conversationnelle permettant à l'apprenant d'interagir naturellement avec le système.

— **Intégration PyPDF2 et OCR** : Pour l'extraction de contenu à partir de documents PDF, permettant d'analyser les ressources pédagogiques et d'enrichir le graphe de connaissances.




FIGURE 4 – Technologies utilisées

## 4.2. Architecture du système

Le système se compose de plusieurs modules qui gèrent les connaissances de l'apprenant, les prérequis et leur poids, tout en utilisant les flags `KNOWN` pour ajuster les interactions :

— **Module de gestion du graphe de connaissances** : Responsable de la création et mise à jour du graphe stocké dans Neo4j, en marquant chaque concept avec un flag `KNOWN` et en associant des prérequis avec des poids compris entre 0 et 1.

— **Module d'évaluation des connaissances** : Ce module génère des questions pour évaluer la maîtrise des concepts. En fonction des réponses de l'apprenant, le flag `KNOWN` est mis à jour et les prérequis avec des poids élevés sont priorisés dans les explications suivantes.

— **Module d'analyse de documents** : Après le téléchargement des PDF par l'apprenant, le système extrait les concepts mentionnés et les compare avec l'état du graphe de connaissances. Les flags `KNOWN` et les poids des prérequis guident la génération des explications.

— **Module d'alignement du LLM** : Le LLM utilise le profil actualisé de l'apprenant, incluant les flags `KNOWN` et les relations de prérequis pondérées, pour générer des explications contextualisées et adaptées au niveau de l'utilisateur.

— **Interface conversationnelle** : Permet à l'apprenant d'interagir avec le système. Elle présente des explications personnalisées et recueille des retours pour ajuster le flag `KNOWN` de chaque concept en fonction des interactions et des performances.

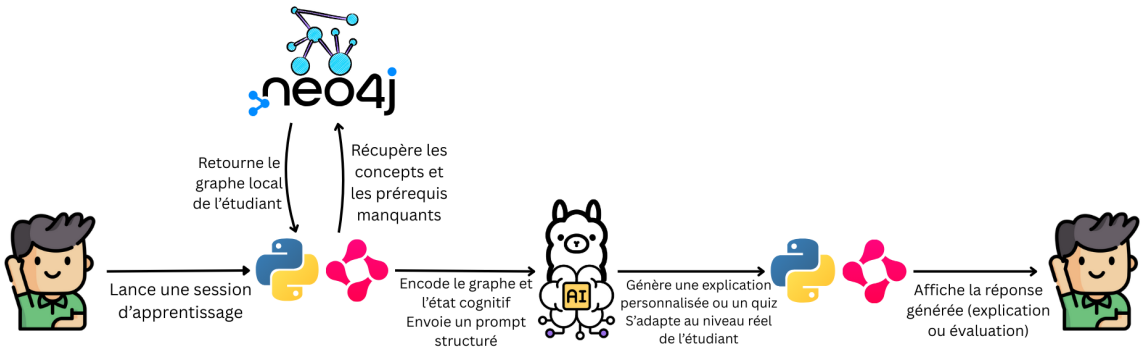



FIGURE 5 – Schéma global du système

### 4.3. Fonctionnalités implémentées

Notre système intègre plusieurs fonctionnalités concrètes permettant une adaptation dynamique des contenus générés à l'état cognitif de chaque apprenant :

- **Évaluation interactive des prérequis** : Le système interroge Neo4j pour identifier les concepts non encore maîtrisés (`KNOWN = 0`). L'apprenant est invité à répondre à des questions de type « connaissez-vous ce concept ? », permettant de mettre à jour son graphe personnel.
- **Génération d'explications conditionnelles** : Lorsqu'un concept est inconnu, le système récupère ses prérequis pondérés dans le graphe et demande au LLM, via un prompt, d'expliquer d'abord ces prérequis avant de traiter le concept cible. Cela garantit une progression adaptée au profil de l'utilisateur.
- **Encodage textuel du graphe dans le prompt** : Les relations entre concepts extraites de Neo4j sont encodées sous forme de texte puis intégrées dans le prompt envoyé au LLM. Cette approche, inspirée de l'article *Talk Like a Graph*, améliore la qualité du raisonnement du modèle.
- **Few-shot prompting simplifié** : Des exemples d'explications ou de quiz sont insérés dans certains prompts pour guider le LLM, améliorant ainsi la structuration et la clarté des contenus générés.
  - **Pourquoi c'est utile ?**
    - Les LLMs peinent à raisonner sur des graphes complexes sans aide.
    - Donner quelques exemples représentatifs améliore fortement leur performance, surtout pour des tâches comme l'identification des prérequis ou le raisonnement chaîné.
    - L'effet est particulièrement positif lorsque les graphes sont encodés de façon claire et concise (ex. : encodage incident) [1].

  Des études montrent que combiner un bon encodage textuel du graphe avec du few-shot prompting augmente la précision de tâches de raisonnement jusqu'à +60% [1].
- **Génération automatique de quiz** : Après chaque explication, un quiz Vrai/Faux est automatiquement généré par le LLM au format JSON. La validation de ce quiz permet de mettre à jour le statut du concept (`KNOWN = 1`) dans le graphe.
- **Analyse de documents PDF** : L'apprenant peut charger un fichier PDF. Le système en extrait le contenu (texte ou via OCR), l'indexe avec Chroma, puis utilise un retriever pour poser des questions sur le contenu ou détecter les concepts abordés.
- **Mise à jour du graphe basée sur les interactions** : Chaque réponse correcte à un quiz ou confirmation d'un concept connu déclenche une mise à jour dans Neo4j, modifiant dynamiquement le graphe de connaissances de l'apprenant.

## 5. Expérimentations et résultats

Bien que notre système soit encore en phase de développement, nous avons pu formuler des observations et comparer théoriquement les bénéfices attendus par rapport à un LLM standard. Cette section présente notre protocole expérimental envisagé, ainsi qu'une analyse comparative basée sur le fonctionnement interne du système et des cas simulés.

## 5.1. Protocole expérimental simulé

Pour évaluer notre approche, nous avons conçu un protocole expérimental simulé pouvant être réalisé avec un groupe d'apprenants dans un futur proche. Il s'articule autour des étapes suivantes :

— **Constitution du graphe de connaissances** : Construction d'un graphe orienté et pondéré dans un domaine ciblé (par exemple : machine learning), comprenant 35 concepts et plus de 200 relations de prérequis.

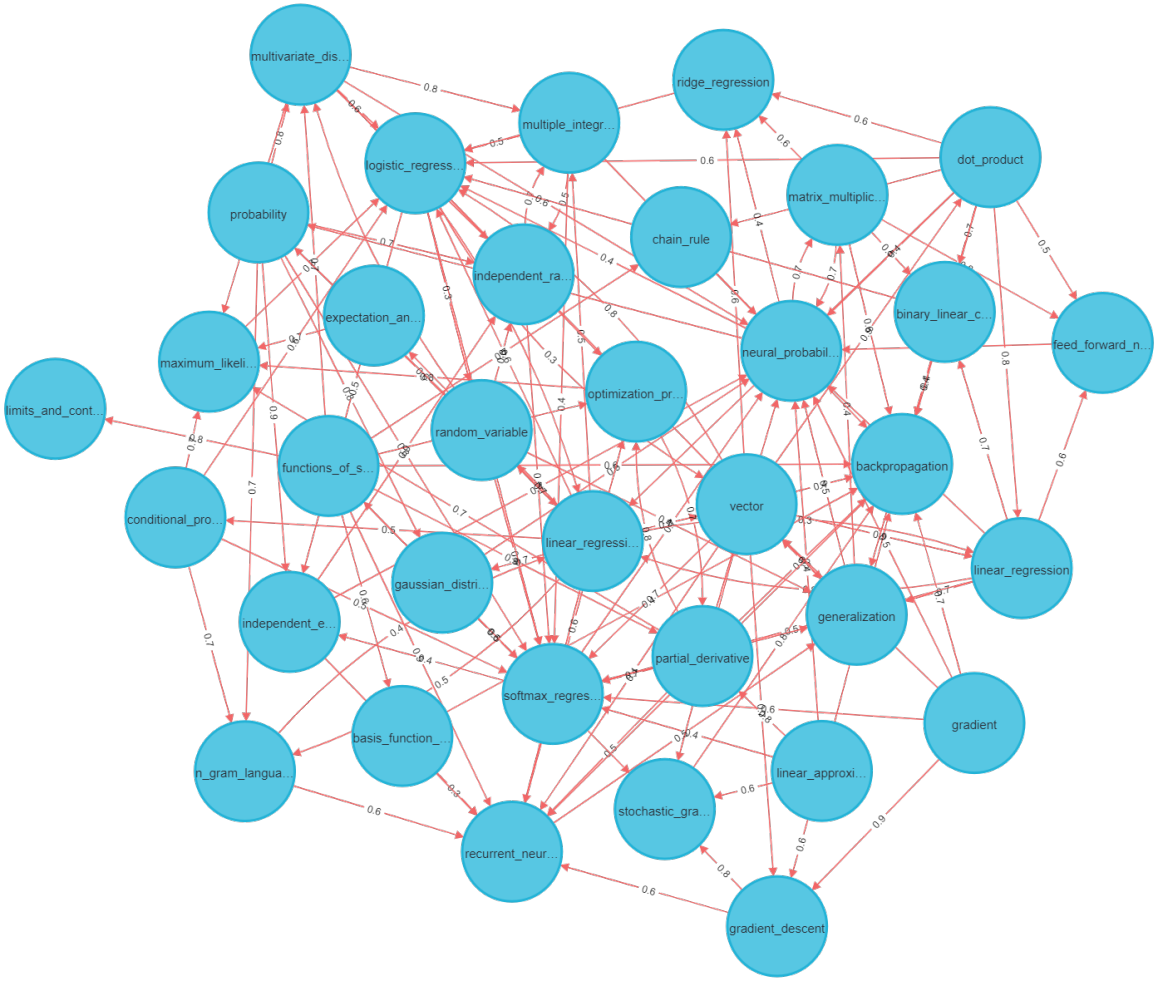



FIGURE 6 – Graphe de prérequis de Machine Learning

— **Modélisation des apprenants** : Chaque étudiant est associé à une vue personnalisée du graphe où chaque concept possède un attribut `KNOWN`, mis à jour dynamiquement après un quiz ou une évaluation interactive.

— **Interface utilisateur** : Illustration du quiz diagnostique initial et de l'évaluation adaptative après explication.

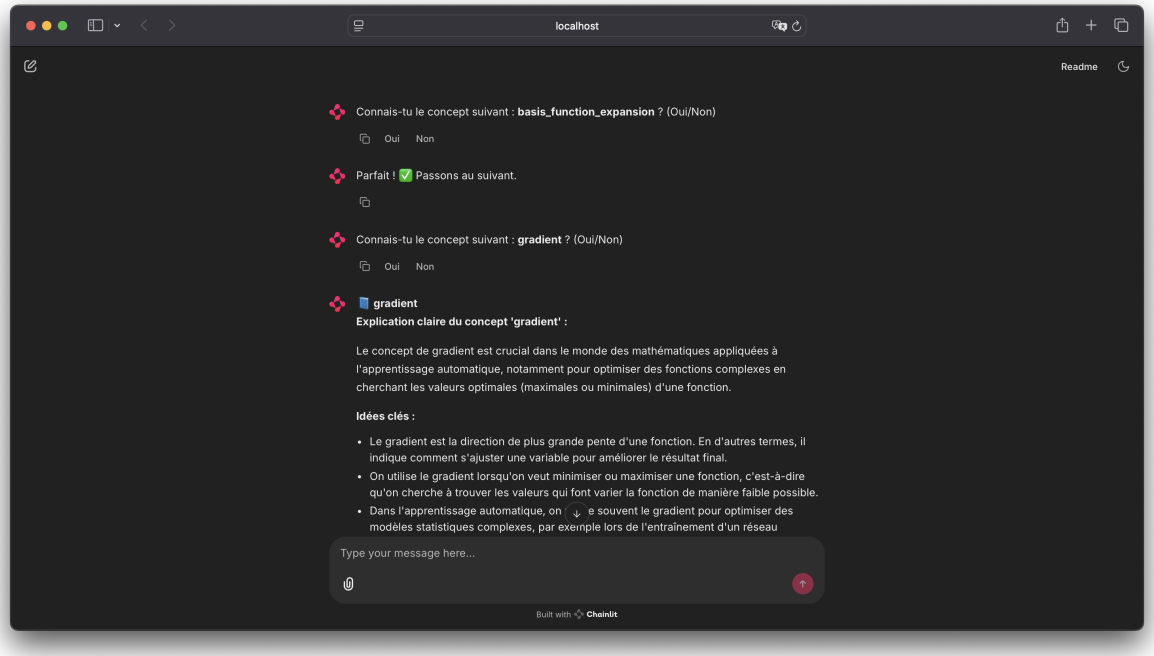



FIGURE 7 – Quiz diagnostique initial et explication des prérequis inconnus

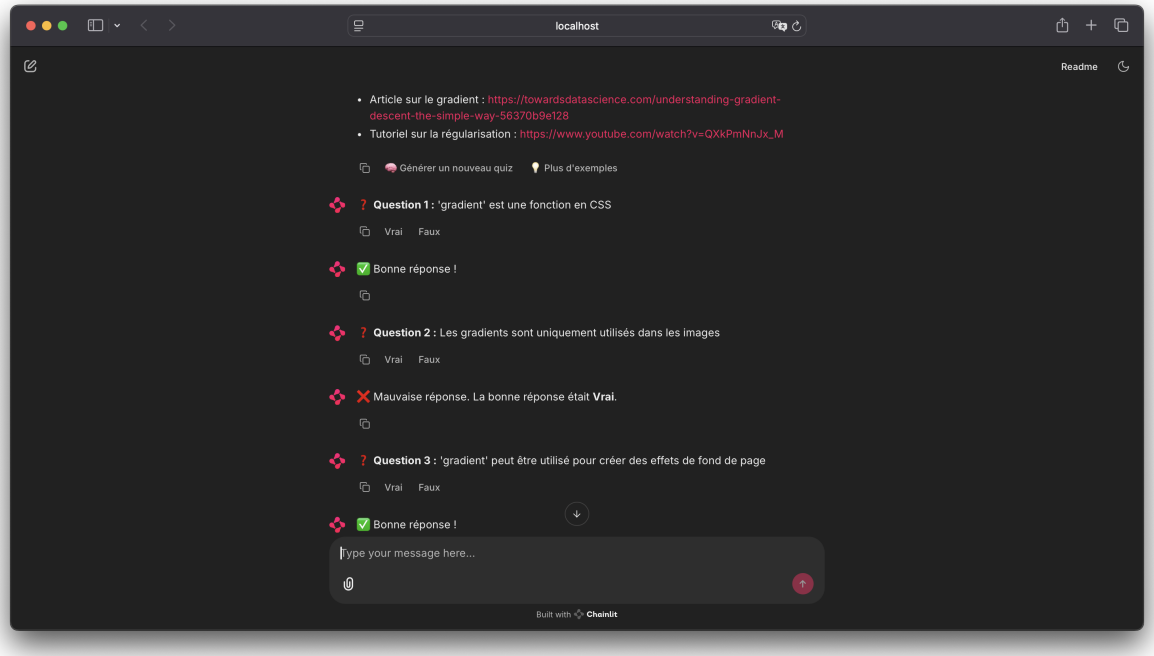



FIGURE 8 – Quiz test après l'explication adaptée

— **Comparaison des approches** :

  — **LLM standard** : Génération d'explications à partir d'un texte ou d'une question, sans tenir compte du niveau réel de l'étudiant.

  — **Notre système aligné** : Utilisation du graphe de prérequis pour guider dynamiquement le LLM, en

intégrant uniquement les concepts maîtrisés et les prérequis nécessaires dans le prompt.

— **Éléments évalués** :
  — Cohérence des explications par rapport au niveau réel de l'étudiant.
  — Progressivité dans l'introduction des concepts.
  — Réduction des erreurs liées à des prérequis manquants.

## 5.2. Résultats observés et comparatifs

Les différences de comportement entre un LLM standard et notre approche orientée graphe ont été observées sur plusieurs aspects clés :

| **Critère** | **LLM standard** | **Notre système (KG + LLM)** |
|---|---|---|
| Prise en compte du niveau étudiant | Aucune | Oui (via `KNOWN` et prérequis) |
| Structuration des explications | Variable, parfois hors de portée | Guidée par les prérequis |
| Cohérence des chaînes logiques | Parfois incohérente | Contrôlée via le graphe orienté |
| Progressivité | Aléatoire | Ajustée dynamiquement |
| Personnalisation | Générique | Personnalisée par graphe individuel |

TABLE 1 – Comparatif entre un LLM standard et notre système aligné

De plus, notre système est capable :

— D'identifier les prérequis manquants d'un concept demandé.
— De générer un quiz ciblé sur ces lacunes.
— D'expliquer d'abord les prérequis avant de traiter le concept cible.

Ces capacités ne sont pas possibles avec un LLM utilisé seul.

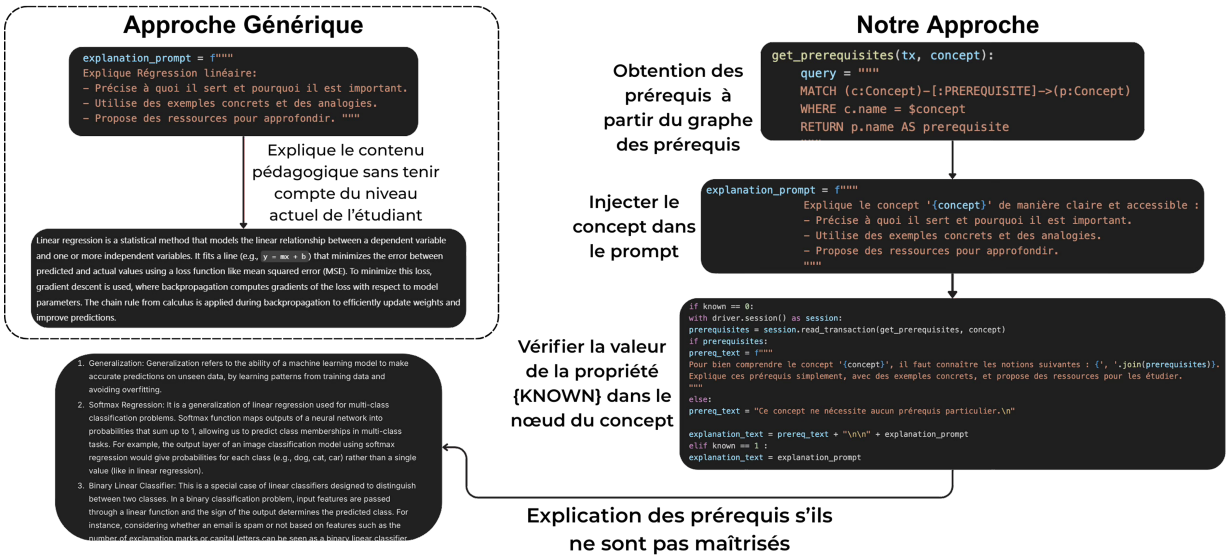



FIGURE 9 – Approche d'un LLM Standard VS Notre Approche

### 5.3. Analyse qualitative

Les retours d'utilisation sur des cas simulés et interactifs ont permis d'identifier plusieurs bénéfices notables :

— **Meilleure clarté des explications** : L'introduction préalable des prérequis réduit les confusions et facilite la compréhension des concepts complexes.

— **Sentiment de progression logique** : Les apprenants perçoivent une montée en compétence structurée et cohérente.

— **Pertinence des contenus générés** : Le LLM, guidé par le graphe, évite d'aborder des notions non maîtrisées par l'étudiant, ce qui limite les hallucinations et les erreurs de raisonnement.

Toutefois, plusieurs limites ont également été relevées :

— **Qualité des prompts sensible à l'encodage du graphe** : Une mauvaise structuration textuelle des relations peut dégrader la pertinence des explications.

— **Présence de redondances** : Si le modèle est trop fortement guidé, des répétitions peuvent apparaître dans les explications successives.

— **Calibration du score de confiance (`KNOWN`) perfectible** : Les seuils actuels nécessitent des ajustements pour mieux refléter la maîtrise réelle de l'apprenant.

## 6. Discussion et limitations

L'analyse des résultats obtenus nous permet d'identifier les forces et les limites de notre approche, ainsi que des pistes d'amélioration pour les travaux futurs.

### 6.1. Points forts

Notre système présente plusieurs avantages notables :

— **Personnalisation effective** : L'alignement du LLM avec le graphe de connaissances permet une adaptation précise des explications au profil individuel de l'apprenant.

— **Évolutivité du modèle apprenant** : La mise à jour dynamique du profil de connaissances permet au système de s'adapter à la progression de l'apprenant.

— **Transparence du processus** : L'utilisation explicite du graphe de connaissances rend le processus d'adaptation plus compréhensible et contrôlable que les approches boîtes noires.

— **Indépendance relative du domaine** : Bien que notre prototype ait été testé dans un domaine spécifique, l'architecture proposée est transposable à d'autres domaines d'apprentissage.

### 6.2. Limitations identifiées

Malgré ces avantages, notre approche présente plusieurs limitations qu'il convient de reconnaître :

— **Coût de construction du graphe** : La création initiale d'un graphe de connaissances de qualité reste un processus exigeant en temps et en expertise.

— **Précision de l'évaluation des connaissances** : L'évaluation automatique des connaissances de l'apprenant reste imparfaite, avec des risques de faux positifs et de faux négatifs.

— **Dépendance aux capacités du LLM** : La qualité des explications générées dépend fondamentalement des capacités du modèle de langage sous-jacent.

— **Gestion des styles d'apprentissage** : Notre approche actuelle se concentre principalement sur les connaissances préalables, sans prendre en compte d'autres facteurs comme les styles d'apprentissage préférentiels.

— **Passage à l'échelle** : L'application de notre approche à des domaines très vastes ou interdisciplinaires pose des défis en termes de complétude et de cohérence du graphe de connaissances.

## 7. Perspectives

### 7.1. Synthèse des contributions

Ce travail a exploré l'alignement des grands modèles de langage (LLMs) avec les représentations mentales des apprenants grâce à l'utilisation de graphes de connaissances. Nos principales contributions sont :

— Une méthodologie pour la construction et l'exploitation de graphes de connaissances représentant à la fois le domaine d'apprentissage et le profil de l'apprenant.

— Une approche d'alignement des LLMs permettant de générer des explications adaptées au niveau de connaissance spécifique de chaque apprenant.

— Un prototype fonctionnel démontrant la faisabilité et l'efficacité de cette approche dans un contexte éducatif.

— Des résultats expérimentaux suggérant une amélioration significative de l'efficacité de l'apprentissage grâce à la personnalisation des explications.

Ces contributions ouvrent la voie à des systèmes d'apprentissage plus intelligents et personnalisés, capables de s'adapter aux besoins spécifiques de chaque apprenant.

### 7.2. Perspectives futures

Plusieurs directions prometteuses pourraient être explorées pour étendre et améliorer ce travail :

— **Automatisation de la construction du graphe** : Développement de méthodes plus automatisées pour extraire des graphes de connaissances à partir de ressources pédagogiques existantes.

— **Intégration de dimensions affectives** : Prise en compte non seulement des connaissances, mais aussi des facteurs motivationnels et émotionnels dans l'adaptation des explications.

— **Approches multimodales** : Extension du système pour générer des explications combinant texte, visualisations et autres modalités adaptées aux préférences de l'apprenant.

— **Évaluation à long terme** : Études longitudinales pour évaluer l'impact de notre approche sur l'apprentissage à long terme et le transfert de connaissances.

— **Co-construction du graphe** : Développement d'approches permettant aux apprenants de contribuer à l'enrichissement du graphe de connaissances, reflétant leur propre compréhension du domaine.

## 8. Conclusion

En conclusion, l'alignement des LLMs avec les graphes de connaissances représente une approche prometteuse pour personnaliser l'apprentissage à l'ère de l'intelligence artificielle. Notre travail constitue une première étape dans cette direction, avec de nombreuses opportunités d'amélioration et d'extension dans des travaux futurs.

# Références


[1] Bahare Fatemi, Jonathan Halcrow, Bryan Perozzi (2023). *Talk like a Graph : Encoding Graphs for Large Language Models*. arXiv preprint arXiv :2310.04560v1.

[2] Zhao, W., Eger, S., Gurevych, I. (2023). *A Survey on Large Language Models for Education*. arXiv preprint arXiv :2312.10530.

[3] Aytekin, M. C., & Saygın, Y. (2024). *ACE : AI-Assisted Construction of Educational Knowledge Graphs with Prerequisite Relations*. Journal of Educational Data Mining, 16(2), 85–114. https://doi.org/10.5281/zenodo.1425089

[4] Kasneci, E., Sessler, K., Küchemann, S., Bannert, M., Dementieva, D., Fischer, F., Gasser, U., Groh, G., Günnemann, S., Hüllermeier, E., et al. (2023). *ChatGPT for good ? On opportunities and challenges of large language models for education*. Learning and Individual Differences, 103, 102274.

[5] Novak, J. D., & Gowin, D. B. (1984). *Learning how to learn*. Cambridge University Press.

[6] Bartlett, F. C. (1932). *Remembering : A study in experimental and social psychology*. Cambridge University Press.

[7] Johnson-Laird, P. N. (1983). *Mental models : Towards a cognitive science of language, inference, and consciousness*. Harvard University Press.

[8] Hogan, A., Blomqvist, E., Cochez, M., d'Amato, C., Melo, G. D., Gutierrez, C., Kirrane, S., Gayo, J. E. L., Navigli, R., Neumaier, S., et al. (2021). *Knowledge graphs*. ACM Computing Surveys, 54(4), 1-37.

[9] Ye, P., Han, X., Lin, B., Liu, Q., Tong, Z., Wei, F. (2023). *Leveraging Knowledge Graphs for Zero-Shot LLM-Based Learning*. Proceedings of the AAAI Conference on Artificial Intelligence, 37(8), 11124-11132.

[10] Graesser, A. C., Chipman, P., Haynes, B. C., & Olney, A. (2005). *AutoTutor : An intelligent tutoring system with mixed-initiative dialogue*. IEEE Transactions on Education, 48(4), 612-618.

[11] Yu, W., Luo, M., Zhou, P., Si, C., Zhou, Y., Wang, X., Wu, J., Chen, X. (2020). *Curriculum Learning : A Survey*. Machine Learning, 1-40.

[12] Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q., Zhou, D. (2022). *Chain of thought prompting elicits reasoning in large language models*. Advances in Neural Information Processing Systems, 35, 24824-24837.

[13] Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., et al. (2022). *Training language models to follow instructions with human feedback*. Advances in Neural Information Processing Systems, 35, 27730-27744.

[14] Christiano, P. F., Leike, J., Brown, T., Martic, M., Legg, S., & Amodei, D. (2017). *Deep reinforcement learning from human preferences*. Advances in Neural Information Processing Systems, 30.

[15] Bai, Y., Kadavath, S., Kundu, S., Askell, A., Kernion, J., Jones, A., Chen, A., Goldie, A., Mirhoseini, A., McKinnon, C., et al. (2022). *Constitutional AI : Harmlessness from AI Feedback*. arXiv preprint arXiv :2212.08073.

[16] Kumar, A., Kalwani, N., Li, J., Gupta, S., West, P. (2023). *Supporting Education with Large Language Models : Opportunities and Alignment Challenges*. arXiv preprint arXiv :2308.05591.

[17] Chen, X., Liu, Q., Li, S., Du, Y., Yu, W., & Wu, J. (2023). *Adaptive Curriculum Learning with Large Language Models for Educational Data Mining*. arXiv preprint arXiv :2310.04560.

[18] Sun, H., Zhang, Y., Wang, S., & Tan, M. (2025). *Educational Knowledge Graph Enhanced Large Language Models*. arXiv preprint arXiv :2503.13340.

[19] Sharath, S., Pardos, Z., & Chuang, I. (2024). *Embedding Knowledge Graphs in Educational Data Mining*. Journal of Educational Data Mining (JEDM), 16(1).

[20] Lee, S., Ryu, J., Choi, J., & Hwang, S. (2022). *Prompt Engineering for Educational Applications of Large Language Models*. arXiv preprint arXiv :2210.12228.

# Glossaire

**LLM** *Large Language Model* - Modèle de langage de grande taille, système d'IA entraîné sur d'immenses corpus de textes capables de générer du contenu linguistique cohérent et contextuel.

**Graphe de connaissances** Structure de données qui représente les connaissances sous forme de graphe orienté où les nœuds sont des concepts et les arêtes des relations sémantiques entre ces concepts.

**Prompt Engineering** Technique consistant à formuler et structurer précisément les instructions données à un LLM pour optimiser ses réponses selon des objectifs spécifiques.

**Fine-tuning** Processus d'ajustement des paramètres d'un modèle préentraîné sur un ensemble de données spécifiques à une tâche pour améliorer ses performances sur cette tâche.

**RLHF** *Reinforcement Learning from Human Feedback* - Apprentissage par renforcement à partir de feedback humain, méthode pour affiner le comportement d'un modèle en utilisant des évaluations humaines.

**Constitutional AI** Approche visant à encadrer le comportement des modèles d'IA par un ensemble de principes prédéfinis, similaires à une constitution.

**Cartes conceptuelles** Représentations graphiques organisées montrant les relations entre différents concepts, utilisées comme outils pédagogiques.

**Théorie des schémas** Théorie cognitive suggérant que la connaissance est organisée en unités structurées (schémas) qui facilitent la compréhension et l'assimilation de nouvelles informations.

**Modèles mentaux** Représentations internes simplifiées que les individus construisent pour comprendre et interagir avec le monde extérieur.

**Curriculum Learning** Stratégie d'apprentissage qui consiste à organiser l'acquisition des connaissances selon une progression de difficulté croissante.

**ACE** *AI-Assisted Construction of Educational Knowledge Graphs with Prerequisite Relations* - Algorithme d'assistance à la construction de graphes de connaissances éducatifs avec relations de prérequis.

**CSR** *Concept Semantic Relatedness* - Mesure quantifiant la proximité sémantique entre deux concepts dans un texte.

**CER** *Concept Embedding Relatedness* - Évaluation de la proximité de deux concepts dans un espace vectoriel d'embeddings.

**PREREQUISITE** Relation dans un graphe de connaissances indiquant qu'un concept doit être maîtrisé avant d'aborder un autre concept.

**KNOWN** Attribut binaire dans un graphe de connaissances indiquant si un concept est maîtrisé (1) ou non (0) par un apprenant.

**Few-shot prompting** Technique consistant à inclure quelques exemples dans les instructions données à un LLM pour guider sa génération.

**Intelligent Tutoring Systems** Systèmes informatiques conçus pour fournir un enseignement personnalisé sans intervention humaine directe.

**Knowledge Forest** Approche utilisant des graphes pour représenter et exploiter des structures de connaissances dans un contexte éducatif.

**Embedding** Représentation vectorielle d'un mot, phrase ou concept dans un espace multidimensionnel, capturant ses caractéristiques sémantiques.

**Chain of thought** Technique de prompting encourageant un LLM à décomposer son raisonnement en étapes intermédiaires explicites.

**Neo4j** Système de gestion de base de données orientée graphe permettant de stocker et interroger efficacement des données structurées en graphe.

**LangChain** Framework Python facilitant le développement d'applications basées sur des LLMs en permettant de créer des chaînes de traitement.

**Chainlit** Bibliothèque Python pour créer rapidement des interfaces conversationnelles basées sur des LLMs.

**Ollama** Outil permettant d'exécuter localement des modèles de langage open-source.

**Talk Like a Graph** Approche consistant à encoder textuellement les informations d'un graphe pour les intégrer dans un prompt de LLM.

# Annexes

## Diagramme de séquence

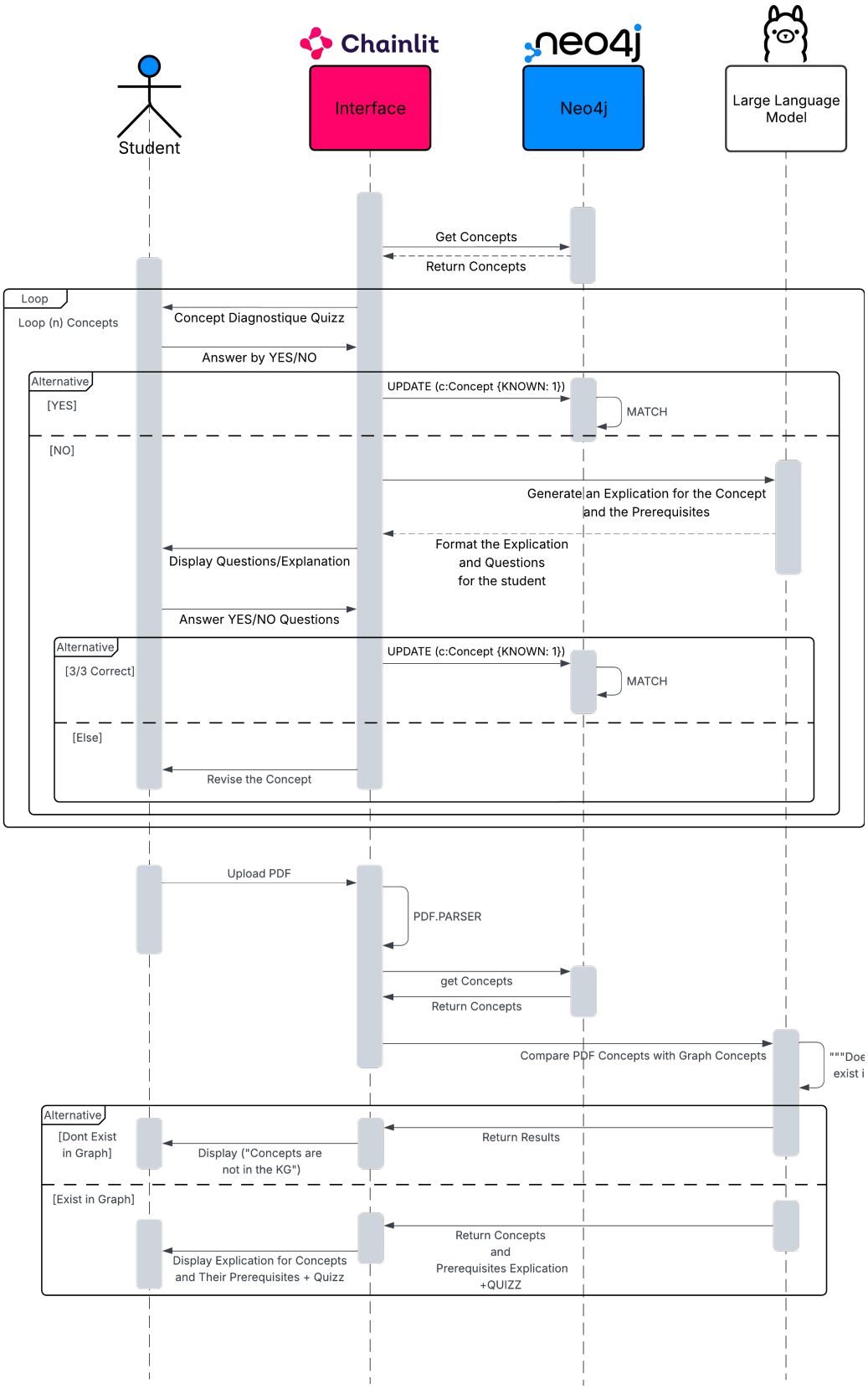



FIGURE 10 – Diagramme de séquence de notre système

### Code Python du système

```python
# Fonction lecture PDF
def extract_text_from_pdf(file_path):
    """
    Extrait le texte du fichier PDF. Si le PDF contient uniquement des images, l'OCR est utilisé pour extraire le texte des images.
    Si le PDF contient à la fois du texte et des images, le texte est extrait et l'OCR est appliqué aux pages sans texte.

    :param file_path: Chemin vers le fichier PDF.
    :return: Le texte extrait du PDF.
    """
    pdf = PyPDF2.PdfReader(file_path)
    pdf_text = ""
    images = convert_from_path(file_path)
    ocr_text = ""

    for page_number, page in enumerate(pdf.pages):
        # Tenter d'extraire le texte
        text = page.extract_text()
        if text:
            pdf_text += text
        else:
            # Si aucun texte n'est extrait, appliquer l'OCR sur l'image de cette page
            print(f"Pas de texte trouvé sur la page {page_number + 1}, tentative d'OCR.")
            img = images[page_number]
            ocr_text += pytesseract.image_to_string(img)

    full_text = pdf_text + "\n" + ocr_text
    # Si le texte extrait est vide, retourner une indication
    if not full_text.strip():
        return None  # Indique que le PDF ne contient pas de texte
    return full_text


# ----------------------------
# Fonctions Graph
# ----------------------------

def get_prerequisites(tx, concept):
    query = """
    MATCH (c:Concept)-[:PREREQUISITE]->(p:Concept)
    WHERE c.name = $concept
    RETURN p.name AS prerequisite
    """
    result = tx.run(query, concept=concept)
    return [record["prerequisite"] for record in result]

```

```
46  def get_all_concepts(tx):
47      query = """
48      MATCH (c:Concept)
49      WHERE c.name IS NOT NULL
50      RETURN c.name AS concept
51      ORDER BY rand()
52      """
53      result = tx.run(query)
54      return [record["concept"] for record in result]
55
56
57  # ---------------------------
58  # Chat start
59  # ---------------------------
60  @cl.on_chat_start
61  async def on_chat_start():
62      with driver.session() as session:
63          concepts = session.read_transaction(get_random_concepts)
64
65      if not concepts:
66          await cl.Message("Aucun concept trouvé dans la base de données.").send()
67          await handle_pdf_upload()
68          return
69
70      cl.user_session.set("concepts", concepts)
71      cl.user_session.set("current_index", 0)
72
73      await ask_concept_question()
74
75
76  # ---------------------------
77  # Étape 1 : Demande initiale
78  # ---------------------------
79
80  async def ask_concept_question():
81      concepts = cl.user_session.get("concepts")
82      index = cl.user_session.get("current_index")
83
84      if index >= len(concepts):
85          await cl.Message("Tu as terminé tous les concepts ! Place maintenant à l'analyse du
        ↪  PDF.").send()
86          await handle_pdf_upload()
87          return
88
89      concept = concepts[index]
90      await cl.Message(
91          content=f"Connais-tu le concept suivant : **{concept}** ? (Oui/Non)",
92          actions=[
93              cl.Action(name="yes", label="Oui", payload={"known": True}),
```

```
94              cl.Action(name="no", label="Non", payload={"known": False})
95          ]
96      ).send()
97
98
99  # ----------------------------
100 # Si le concept est connu
101 # ----------------------------
102 @cl.action_callback("yes")
103 async def handle_known_concept(action):
104     # Récupérer le concept actuel
105     concepts = cl.user_session.get("concepts")
106     index = cl.user_session.get("current_index")
107     concept = concepts[index]
108
109     # Mise à jour de la propriété 'known' à 1 dans Neo4j
110     try:
111         with driver.session() as session:
112             session.run("""
113                 MATCH (c:Concept)
114                 WHERE c.name = $concept
115                 SET c.known = 1
116             """, concept=concept)
117         await cl.Message("Parfait ! Passons au suivant.").send()
118     except Exception as e:
119         await cl.Message(f"Erreur lors de la mise à jour du concept : {e}").send()
120
121     # Passer au concept suivant
122     cl.user_session.set("current_index", cl.user_session.get("current_index") + 1)
123
124     await ask_concept_question()
125
126
127 # ----------------------------
128 # Si inconnu -> Explication + Quiz
129 # ----------------------------
130 @cl.action_callback("no")
131 async def handle_unknown_concept(action):
132     concepts = cl.user_session.get("concepts")
133     index = cl.user_session.get("current_index")
134     concept = concepts[index]
135
136     with driver.session() as session:
137         prerequisites = session.read_transaction(get_prerequisites, concept)
138
139     prereq_text = f"""
```

```
    Pour bien comprendre le concept '{concept}', il est important de se concentrer uniquement sur les prérequis les plus pertinents et directement liés à ce concept. Parmi les notions suivantes : {', '.join(prerequisites)}, identifie uniquement celles qui sont étroitement liées à '{concept}'.
    Explique uniquement ces prérequis de manière simple, avec des exemples concrets, et propose des ressources utiles pour les étudier."""

    concept_text = f"""
    Explique ensuite le concept '{concept}' de manière claire et accessible :
    - Décris son but, son utilité et dans quel contexte il est important.
    - Utilise des exemples concrets et des analogies pour faciliter la compréhension.
    - Suggère quelques ressources (articles, tutoriels, vidéos, etc.) pour aller plus loin et mieux maîtriser ce concept.
    """

    quiz_prompt = (
        f"Génère 3 questions Vrai/Faux avec les bonnes réponses, au format JSON comme ceci :\n"
        f"[{{'question': '...', 'answer': 'Vrai'}}, ...] sur le concept '{concept}'."
    )

    try:
        explanation = await cl.make_async(llm.invoke)(prereq_text + "\n\n" + concept_text)
        quiz_json = await cl.make_async(llm.invoke)(quiz_prompt)

        # Vérification du format du JSON
        try:
            quiz_data = json.loads(quiz_json.strip())
            if not isinstance(quiz_data, list) or not all(isinstance(q, dict) and 'question' in q and 'answer' in q for q in quiz_data):
                raise ValueError("Le format du quiz généré est incorrect.")
        except (json.JSONDecodeError, ValueError) as e:
            await cl.Message(f"Erreur dans le format du quiz généré : {str(e)}").send()
            cl.user_session.set("current_index", cl.user_session.get("current_index") + 1)
            await ask_concept_question()
            return

    except Exception as e:
        await cl.Message(f"Erreur pendant la génération : {e}").send()
        cl.user_session.set("current_index", cl.user_session.get("current_index") + 1)
        await ask_concept_question()
        return

    # Afficher explication
    await cl.Message(f"**Explication de {concept}**\n\n{explanation.strip()}").send()

    # Sauvegarde du quiz et gestion de la session
    cl.user_session.set("current_quiz", quiz_data)
    cl.user_session.set("quiz_index", 0)
```

```
182     cl.user_session.set("quiz_score", 0)
183
184     await send_quiz_question()
185
186
187 # ----------------------------
188 # Envoyer une question du quiz
189 # ----------------------------
190 async def send_quiz_question():
191     quiz = cl.user_session.get("current_quiz")
192     index = cl.user_session.get("quiz_index")
193
194     if index >= len(quiz):
195         score = cl.user_session.get("quiz_score")
196         total = len(quiz)
197
198         result_msg = f"Tu as bien compris ! Score : {score}/{total}" if score == total else
        ↪  f"Tu as eu {score}/{total}. Continue de réviser !"
199         await cl.Message(result_msg).send()
200
201         # Si toutes les questions sont répondues correctement, mettre à jour le concept
        ↪  comme "connu"
202         if score == total:
203             concepts = cl.user_session.get("concepts")
204             current_index = cl.user_session.get("current_index")
205             if current_index < len(concepts):
206                 concept = concepts[current_index]
207                 try:
208                     with driver.session() as session:
209                         session.run("""
210                             MATCH (c:Concept)
211                             WHERE c.name = $concept
212                             SET c.known = 1
213                         """, concept=concept)
214                     await cl.Message(f"Le concept '{concept}' est maintenant marqué comme
                    ↪  connu.").send()
215                 except Exception as e:
216                     await cl.Message(f"Erreur lors de la mise à jour du concept :
                    ↪  {e}").send()
217
218         # Passer au concept suivant
219         cl.user_session.set("current_index", cl.user_session.get("current_index") + 1)
220
221         # Vérifier si tous les concepts ont été vus
222         concepts = cl.user_session.get("concepts")
223         current_index = cl.user_session.get("current_index")
224
225         if current_index >= len(concepts):
```

```
226             await cl.Message("Tu as terminé tous les concepts ! Place maintenant à l'analyse
                ↪  du PDF.").send()
227             await handle_pdf_upload()  # Lance l'analyse du PDF
228         else:
229             await ask_concept_question()
230         return
231
232     question = quiz[index]["question"]
233     await cl.Message(
234         content=f"**Question {index+1} :** {question}",
235         actions=[
236             cl.Action(name="true", label="Vrai", payload={"response": "Vrai"}),
237             cl.Action(name="false", label="Faux", payload={"response": "Faux"})
238         ]
239     ).send()
240
241
242 # ----------------------------
243 # Réponse au quiz
244 # ----------------------------
245 @cl.action_callback("true")
246 async def answer_true(action):
247     await handle_quiz_response("Vrai")
248
249 @cl.action_callback("false")
250 async def answer_false(action):
251     await handle_quiz_response("Faux")
252
253 async def handle_quiz_response(user_answer):
254     quiz = cl.user_session.get("current_quiz")
255     index = cl.user_session.get("quiz_index")
256     correct = quiz[index]["answer"]
257
258     if user_answer.lower() == correct.lower():
259         await cl.Message("Bonne réponse !").send()
260         cl.user_session.set("quiz_score", cl.user_session.get("quiz_score") + 1)
261     else:
262         await cl.Message(f"Mauvaise réponse. La bonne réponse était **{correct}**.").send()
263
264     cl.user_session.set("quiz_index", index + 1)
265     await send_quiz_question()
266
267
268 # ----------------------------
269 # Lancer l'analyse du PDF après la fin des quiz
270 # ----------------------------
271 async def handle_pdf_upload():
272     files = None
273
```

```
274     # Demande d'upload de fichier
275     while files is None:
276         files = await cl.AskFileMessage(
277             content="Veuillez uploader un fichier PDF pour commencer !",
278             accept=["application/pdf"],
279             max_size_mb=100,
280             timeout=180
281         ).send()
282 
283     file = files[0]
284     pdf_text = extract_text_from_pdf(file.path)
285 
286     if not pdf_text:
287         await cl.Message(content="Le PDF téléchargé ne contient pas de texte exploitable.
        ↪  Veuillez essayer avec un autre fichier.").send()
288         return
289 
290     # Découpage du texte
291     text_splitter = RecursiveCharacterTextSplitter(chunk_size=1200, chunk_overlap=50)
292     texts = text_splitter.split_text(pdf_text)
293     metadatas = [{"source": f"{i}-pl"} for i in range(len(texts))]
294 
295     # Embedding & indexation
296     embeddings = OllamaEmbeddings(model="nomic-embed-text")
297     docsearch = await cl.make_async(Chroma.from_texts)(texts, embeddings,
        ↪  metadatas=metadatas)
298 
299     # Mémoire de conversation
300     memory = ConversationBufferMemory(
301         memory_key="chat_history",
302         output_key="answer",
303         chat_memory=ChatMessageHistory(),
304         return_messages=True,
305     )
306 
307     # Création de la chaîne de QA
308     chain = ConversationalRetrievalChain.from_llm(
309         llm=llm,
310         chain_type="stuff",
311         retriever=docsearch.as_retriever(),
312         memory=memory,
313         return_source_documents=True
314     )
315     cl.user_session.set("chain", chain)
316 
317     # Message d'attente pour l'analyse du PDF
318     wait_message = await cl.Message("Analyse du PDF en cours...").send()
319 
320     # Résumé du contenu
```

```
321     summary_prompt = f"""
322     Fais un résumé structuré et concis du texte suivant, en mettant en valeur les concepts
        ↪  clés et idées principales :
323     {pdf_text}
324     """
325     summary = await cl.make_async(llm.invoke)(summary_prompt)
326
327     # Récupération de tous les concepts
328     with driver.session() as session:
329         concepts = session.read_transaction(get_all_concepts)
330     cl.user_session.set("concepts", concepts)
331
332     matched_concepts = set()
333
334     # Détection des concepts mentionnés dans le résumé
335     for concept in concepts:
336         prompt = f"""
337         Analyse ce résumé et détermine s'il mentionne ou traite du concept \"{concept}\".
338         Réponds uniquement par 'Oui' ou 'Non'.
339
340         Résumé :
341         {summary}
342         """
343         try:
344             response = await cl.make_async(llm.invoke)(prompt)
345             if "oui" in response.strip().lower():
346                 matched_concepts.add(concept)
347
348                 with driver.session() as session:
349                     result = session.run("MATCH (c:Concept) WHERE c.name = $concept RETURN
                        ↪  c.known AS known", concept=concept)
350                     known = result.single()["known"]
351
352                 explanation_prompt = f"""
353                 Explique le concept '{concept}' de manière claire et accessible :
354                 - Précise à quoi il sert et pourquoi il est important.
355                 - Utilise des exemples concrets et des analogies.
356                 - Propose des ressources pour approfondir.
357                 """
358
359                 if known == 0:
360                     with driver.session() as session:
361                         prerequisites = session.read_transaction(get_prerequisites, concept)
362
363                     if prerequisites:
364                         prereq_text = f"""
```

```
365                         Pour bien comprendre le concept '{concept}', il est important de se
                            ↪ concentrer uniquement sur les prérequis les plus pertinents et
                            ↪ directement liés à ce concept. Parmi les notions suivantes : {',
                            ↪ '.join(prerequisites)}, identifie uniquement celles qui sont
                            ↪ étroitement liées à '{concept}'.
366                         Explique uniquement ces prérequis de manière simple, avec des
                            ↪ exemples concrets, et propose des ressources utiles pour les
                            ↪ étudier."""
367                     else:
368                         prereq_text = "Ce concept ne nécessite aucun prérequis
                            ↪ particulier.\n"
369
370                     explanation_text = prereq_text + "\n\n" + explanation_prompt
371                 else:
372                     explanation_text = explanation_prompt
373
374                 explanation = await cl.make_async(llm.invoke)(explanation_text)
375                 await cl.Message(f"**Explication de
                    ↪ {concept}**\n\n{explanation.strip()}").send()
376
377         except Exception as e:
378             print(f"Erreur lors de la vérification du concept '{concept}' : {e}")
379
380     await wait_message.remove()
381
382     if matched_concepts:
383         cl.user_session.set("matched_concepts", matched_concepts)
384
385         # Génération du quiz
386         quiz_prompt = f"""
387         Génère 5 questions de quiz à choix multiples (QCM), chacune portant sur un des
            ↪ concepts suivants : {', '.join(matched_concepts)}.
388         Pour chaque question :
389         - Propose une bonne réponse et trois distracteurs plausibles.
390         - N'indique PAS la bonne réponse.
391         - Utilise un format clair comme :
392         **Question 1 :** Quel est le rôle de XYZ ?
393         A. Réponse plausible
394         B. Réponse plausible
395         C. Réponse plausible
396         D. Réponse plausible
397         Adapte la langue du quiz à celle des concepts si besoin.
398         """
399
400         quiz_output = await cl.make_async(llm.invoke)(quiz_prompt)
401         await cl.Message(f"**Quiz basé sur les concepts détectés
            ↪ :**\n\n{quiz_output.strip()}").send()
402     else:
403         await cl.Message("Aucun concept détecté dans ce document.").send()
```

```
404
405     cl.user_session.set("full_pdf_text", pdf_text)
406     await cl.Message("Le PDF a été traité. Vous pouvez maintenant poser vos questions
        ↪  !").send()
```